FAKULTA INFORMATIKY A INFORMAČNÝCH TECHNOLÓGIÍ
Slovenská Technická Univerzita v Bratislave

MILESTONE 2

Backend Documentation & Updated Application Specification

Subject: ZMABT_B
Instructor: Ing. Dominik Šalgovič
Authors: Adam Strelec, Damián Štyrák
Field of study: Computer Science
Academic year: 2025/26

Contents

1 Application Description **2**

2 User Roles **2**
2.1 Manager 2
2.2 Worker 3
2.3 Use Cases 3
2.3.1 Manager 3
2.3.2 Worker 4

3 Acceptance Tests (UAT) **5**

4 Backend Documentation (Milestone 2) **12**
4.1 Repository and Submission Commit 12
4.2 Technology Stack 12
4.3 Architecture 12
4.4 Database Schema 12
4.5 API Endpoints 13
4.6 Real-time Communication 15
4.7 Security 15

1. Application Description

WorkFlow is a cross-platform mobile application (iOS/Android) designed for coordinating field work teams. It replaces inefficient phone - or email-based communication by giving managers an instant, real-time overview of task progress and team locations.

The application allows a manager to create projects and assign tasks within them to workers based on their verified skill certificates. Workers receive push notifications about new assignments, can update task statuses, upload photo documentation directly from the field, and communicate with the manager via a built-in chat. The manager tracks live worker locations on a map and verifies worker qualifications through submitted certificates.

The application supports offline mode — changes made without an internet connection are automatically synchronised once connectivity is restored.

Wireframes and Prototype

Wireframes and the interactive prototype are available at the following link:

https://www.figma.com/design/5Mkvlexdcuru8Y7Si9LxXP/...

Technology Stack

Layer	Technology
Frontend	React Native with Expo (iOS / Android)
Backend	Node.js (Express)
Database	PostgreSQL
Push notifications	FCM (Android) + APNs via FCM (iOS)
Authentication	Google SSO (OAuth 2.0)
Maps	react-native-maps (Google Maps / Apple Maps)

2. User Roles

The application defines two user roles with distinct permissions.

2.1. Manager

The Manager has full control over projects, tasks, and the team:

- Creates, edits, and deletes projects.
- Creates, edits, and deletes tasks within projects (title, description, GPS coordinates, deadline, category, priority).
- Assigns tasks to workers based on their verified skill certificates.
- Views live worker positions and task-site pins on the map.
- Has an overview of all tasks and their current statuses.
- Invites new workers to the application via an invite link.
- Reviews and accepts worker skill certificates.
- Communicates with workers via the built-in chat.
- Receives a push notification when a worker marks a task as *Done*.

2.2. Worker

The Worker has restricted access — only to tasks assigned to them:

- Views the list of assigned tasks within projects.
- Opens task detail: description, category, assigned person, deadline, photos, and status.
- Changes the task status to *In Progress* or *Done*.
- Uploads photo documentation from the camera or gallery.
- Uploads skill certificates for manager review.
- Shares GPS location with the manager (every 10 seconds).
- Communicates with the manager and other workers via chat.
- Receives a push notification when a new task is assigned.
- Can update task status offline; the change syncs after reconnection.

2.3. Use Cases

2.3.1 Manager

UCM1 — Login: The Manager logs into the application using Google SSO or email and password. Upon successful authentication a JWT token is issued and stored securely on the device.

UCM2 — Create, edit, delete a project: The Manager creates a new project by filling in the project title, description, deadline, construction site address, GPS coordinates, and client information (name, phone, email). Existing projects can be edited or permanently deleted. Deleting a project removes all associated tasks and assignments.

UCM3 — Create, edit, delete a task: Within a project the Manager creates tasks by specifying a title, description, category (e.g. Electrical, Heating), priority (Low / Medium / High), and deadline. Tasks can be edited or deleted at any time. Only workers with a verified certificate matching the task category are eligible for assignment.

UCM4 — Assign a task to a worker: The Manager assigns a task to one or more workers. The assignment screen displays available workers filtered by their verified skill certificates and flags workers missing the required qualification as ineligible. Upon assignment the worker receives a push notification.

UCM5 — View all tasks and their statuses: The Manager sees a full list of tasks within a project with their current statuses (Pending, In Progress, Done) and an overall progress indicator showing how many tasks have been completed.

UCM6 — Review and accept worker skill certificates: When a worker submits a certificate, the Manager receives a notification and can view the certificate photo. The Manager either accepts the certificate — making the worker eligible for the corresponding task category — or rejects it.

UCM7 — Team map: The Manager opens a project-specific map showing the live GPS positions of all workers assigned to that project, updated every 10 seconds via WebSocket. Workers are shown with an online/offline indicator.

UCM8 — Chat with a worker: The Manager initiates or continues a real-time conversation with any worker via the built-in chat. Messages are delivered instantly over a

WebSocket connection.

UCM9 — Invite a new worker: The Manager generates a unique invite link from the Worker List screen. The link can be copied and shared with a prospective worker, who uses it to register and join the Manager's team.

UCM10 — Receive notification when a task is completed: When a worker marks a task as Done, the Manager receives a background push notification. Tapping the notification navigates directly to the relevant task.

2.3.2 Worker

UCW1 — Login: The Worker logs into the application using Google SSO or email and password.

UCW2 — View list of assigned tasks: The Worker sees only the projects and tasks assigned to them. Each task card shows the title, category, deadline, and current status.

UCW3 — Open task detail: The Worker opens a task to view the full description, category, assigned person, deadline, priority, and any previously uploaded photos.

UCW4 — Update task status: The Worker changes the task status to *In Progress* or *Done* directly from the task card. The change is reflected in real time on the Manager's view.

UCW5 — Upload a photo to a task: The Worker attaches photo documentation to a task by taking a photo with the camera or selecting one from the gallery. Multiple photos can be uploaded per task.

UCW6 — Upload skill certificates: The Worker submits a certificate of qualification for a specific skill category by uploading a photo of the document. The certificate is sent to the Manager for review and, once accepted, unlocks the Worker for tasks requiring that skill.

UCW7 — Offline task status update: When the device has no internet connection the Worker can still change a task status. The change is saved locally with a pending sync flag and automatically synchronised with the server once connectivity is restored. Conflicts are resolved with a server-wins policy.

UCW8 — Chat with the manager: The Worker sends and receives real-time messages with the Manager. If the device is offline the chat is unavailable and an informative banner is shown.

UCW9 — Receive notification about a new task assignment: When the Manager assigns a task to the Worker, a background push notification is delivered to the Worker's device. Tapping the notification navigates directly to the task detail.

3. Acceptance Tests (UAT)

Positive Tests

UAT 1+: Assign Job to Worker	
Test Case ID	UAT-01+
Description	The manager assigns a job to a worker.
Preconditions	• Manager must be logged in • Manager must have at least one project with at least one worker • Manager must be connected to the internet
Test Scenario	1. Manager opens the Project List and clicks on existing project. 2. A project details are displayed. 3. Manager taps the **Assign Worker** button. 4. A new screen with available workers is shown. 5. Manager selects a worker by clicking on available worker. 6. The assigned worker's name appears in the Project Detail screen.
Output Conditions	The worker is successfully assigned to the project and their name appears in the Project Detail screen.
Result: PASS / FAIL	

UAT 2+: Manager Creates a New Project	
Test Case ID	UAT-02+
Description	The manager creates a new project.
Preconditions	• Manager is logged in • Internet connection is available
Test Scenario	1. The manager opens the application and navigates to the project list. 2. The manager taps *"Add project"*. 3. A pop-up window appears with default project parameters and options to add tasks. 4. The manager sets the project parameters and adds specific tasks and workers. 5. The manager confirms the creation. 6. The new project appears in the project list.
Output Conditions	The project is successfully created and is visible in the project list.
Result: PASS / FAIL	

UAT 3+: Create New Task	
Test Case ID	UAT-03+
Description	The manager creates a new task within an existing project.
Preconditions	• Manager is logged in • Internet connection is available • The project in which the task will be created already exists
Test Scenario	1. The manager opens the application and navigates to the project list. 2. The manager selects the project in which they want to create the task. 3. The manager opens the Job Detail screen. 4. The manager taps *"Add task"*. 5. The manager enters the task name and optionally assigns workers. 6. The manager confirms the creation.
Output Conditions	The task is successfully created and appears in the task list of the project.
Result: PASS / FAIL	

UAT 4+: The Manager Inspects Workers on the Map	
Test Case ID	UAT-04+
Description	The manager inspects the location of a worker assigned to a project using the map.
Preconditions	• Manager is logged in • Internet connection is available • The project to which the worker is assigned already exists • The inspected worker is online
Test Scenario	1. The manager opens the application and navigates to the project list. 2. The manager selects the project to which the worker is assigned. 3. The manager taps the *"View Map"* button. 4. The system displays the map and the list of workers assigned to the project. 5. The manager taps on the selected worker. 6. The worker's current location is displayed on the map.
Output Conditions	The worker's location is displayed on the map.
Result: PASS / FAIL	

UAT 5+: The Worker Sends a Message to the Manager	
Test Case ID	UAT-05+
Description	The worker sends a message to the manager.
Preconditions	• Worker is logged in • Internet connection is available • A chat between the worker and manager already exists
Test Scenario	1. The worker opens the application and navigates to the navigation bar at the bottom of the screen. 2. The worker taps the message icon in the navigation bar. 3. The worker locates the chat with the manager in the chat list and taps on it. 4. The worker writes a message and taps the send icon.
Output Conditions	The message is successfully sent and appears in the chat conversation.
Result: PASS / FAIL	

UAT 6+: The Worker Uploads a Plumber Certificate	
Test Case ID	UAT-06+
Description	The worker uploads a plumber certificate so the manager can see their area of expertise.
Preconditions	• Worker is logged in • Internet connection is available
Test Scenario	1. The worker opens the application and navigates to the navigation bar at the bottom of the screen. 2. The worker taps the profile icon in the navigation bar. 3. The worker taps the *Edit Profile* button. 4. The worker locates the relevant area of expertise and taps *Upload*. 5. The worker selects the certificate file and confirms the choice. 6. The worker taps *Save* to save the profile changes.
Output Conditions	The certificate is uploaded and visible in the worker's profile.
Result: PASS / FAIL	

UAT 7+: The Manager Sends an Invitation to a Worker	
Test Case ID	UAT-07+
Description	The manager invites a worker to be connected in the app.
Preconditions	• Manager is logged in • Internet connection is available
Test Scenario	1. The manager opens the application and navigates to the Workers section. 2. The manager taps the *Invite Worker* button. 3. A pop-up window appears displaying the invite link. 4. The manager copies the link and sends it to the worker.
Output Conditions	The invite link is successfully generated and displayed to the manager in a pop-up window.
Result: PASS / FAIL	

Negative Tests

UAT 1-: Manager Tries to Create a Project with a Duplicate Name	
Test Case ID	UAT-01-
Description	The system prevents the manager from creating a project with a name that is already in use.
Preconditions	• Manager is logged in • Internet connection is available • A project with the same name already exists
Test Scenario	1. The manager opens the application and navigates to the project list. 2. The manager taps *"Add project"*. 3. A pop-up window appears with default project parameters. 4. The manager enters a name that is already in use. 5. The manager confirms the action. 6. An error message appears below the input field. 7. The manager enters a unique name and confirms successfully.
Output Conditions	The manager is notified of the duplicate name and successfully creates the project with a unique name.
Result: PASS / FAIL	

UAT 2-: Worker Tries to Send a Message Without Internet	
Test Case ID	UAT-02-
Description	The system prevents the worker from sending a message when there is no internet connection.
Preconditions	• Worker is logged in • Internet connection is unavailable • A chat between the worker and manager already exists
Test Scenario	1. The worker opens the application and navigates to the navigation bar at the bottom of the screen. 2. The worker taps the message icon in the navigation bar. 3. The worker locates the chat with the manager and taps on it. 4. The worker writes a message and taps the send icon. 5. An error message appears informing the worker that the message could not be sent.
Output Conditions	The message is not sent and an error message is displayed to the worker.
Result: PASS / FAIL	

UAT 3-: Manager Tries to Create a Task Without a Name	
Test Case ID	UAT-03-
Description	The system prevents the manager from creating a task without entering a name.
Preconditions	• Manager is logged in • Internet connection is available • The project in which the task will be created already exists
Test Scenario	1. The manager opens the application and navigates to the project list. 2. The manager selects the project in which they want to create the task. 3. The manager opens the Job Detail screen. 4. The manager taps *"Add task"*. 5. The manager leaves the task name field empty and confirms the creation. 6. An error message appears below the name input field.
Output Conditions	The task is not created and an error message is displayed below the name input field.
Result: PASS / FAIL	

4. Backend Documentation (Milestone 2)

4.1. Repository and Submission Commit

Item	Value
GitHub repository	https://github.com/StyroD/zmabt
Submission commit hash	`5ea53f4a`
Branch	`adam`

4.2. Technology Stack

Layer	Technology
Language	TypeScript (Node.js)
Framework	Express.js
Database	PostgreSQL (via `pg` library, no ORM)
Authentication	JWT (JSON Web Token) + Google OAuth 2.0 (Passport.js)
Real-time	Socket.IO (WebSocket)
File uploads	Multer (multipart/form-data)
Push notifications	FCM (Firebase Cloud Messaging)
API docs	Swagger UI (`/api-docs`)

4.3. Architecture

The backend follows a layered REST architecture running locally on `http://localhost:3000`. All protected endpoints require a Bearer JWT token in the `Authorization` header.

- **Routes** — define HTTP method and path, delegate to query handlers.
- **Queries** — contain business logic and direct SQL via `pg` Pool.
- **Middleware** — JWT authentication guard applied globally (except `/api/auth` and `/api/invite`).
- **Socket** — Socket.IO server authenticated via JWT handshake; each user joins a private room `user:{id}` for targeted real-time message delivery.
- **Services** — FCM push notification helper called after task assignment and status changes.

4.4. Database Schema

The PostgreSQL database uses UUIDs as primary keys. The following tables are created by the migration script:

Table	Purpose
`users`	Both managers and workers; role field distinguishes them
`invite_links`	One-time tokens generated by managers for worker registration
`projects`	Construction projects owned by a manager
`project_workers`	Many-to-many: workers assigned to projects
`project_photos`	Photos attached to a project
`skill_categories`	Certificate/task skill categories (e.g. Electrical, Plumbing)
`certificates`	Worker skill certificates submitted for manager review
`tasks`	Tasks within a project; linked to a skill category
`task_assignments`	Many-to-many: workers assigned to tasks
`task_photos`	Photo documentation attached to a task
`task_status_sync`	Offline status changes queued for sync on reconnect
`worker_locations`	Latest GPS coordinates per worker per project
`messages`	Chat messages between two users
`notifications`	In-app notifications with read status
`audit_logs`	Immutable record of actions for auditing

4.5. API Endpoints

All endpoints (except `/api/auth/*` and `/api/invite/register`) require `Authorization: Bearer <token>`.

Auth — `/api/auth`

Method	Path	Description
POST	`/register`	Register a new user (email + password)
POST	`/login`	Login; returns JWT token
GET	`/google`	Initiate Google OAuth 2.0 flow
GET	`/google/callback`	Google OAuth callback

Users — `/api/users`

Method	Path	Description
POST	`/`	Create user
GET	`/:id`	Get user by ID
PUT	`/:id`	Update user profile (supports avatar upload)
PATCH	`/:id/push-token`	Update FCM push token

Projects — `/api/projects`

Method	Path	Description
POST	`/`	Create a project
GET	`/:id`	Get project by ID
PUT	`/:id`	Update project
DELETE	`/:id`	Delete project
GET	`/manager/:managerId`	Get all projects for a manager
GET	`/:id/workers`	List workers assigned to project
POST	`/:id/workers`	Assign a worker to project
DELETE	`/:id/workers/:workerId`	Remove worker from project

Tasks — `/api/tasks`

Method	Path	Description
POST	`/`	Create a task
GET	`/:id`	Get task by ID
PUT	`/:id`	Update task
DELETE	`/:id`	Delete task
PATCH	`/:id/status`	Update task status (*pending* / *in_progress* / *done*)
POST	`/:id/workers`	Assign workers to a task
GET	`/project/:id`	Get all tasks for a project
GET	`/worker/:workerId`	Get all tasks assigned to a worker

Workers — `/api/workers`

Method	Path	Description
GET	`/:projectId`	Get workers with certificates for a project

Uploads — `/api/uploads`

Method	Path	Description
POST	`/certificate`	Upload a skill certificate (multipart/form-data)
GET	`/certificate/:id`	Download certificate binary by ID

Locations — `/api/locations`

Method	Path	Description
POST	`/`	Update worker GPS location
GET	`/project/:projectId`	Get all worker locations for a project

Messages — `/api/messages`

Method	Path	Description
GET	`/conversations`	List all conversations for the logged-in user
POST	`/`	Send a message (also emits via Socket.IO)
GET	`/:otherUserId`	Get message history with another user

Notifications — `/api/notifications`

Method	Path	Description
GET	`/user/:userId`	Get all notifications for a user
PATCH	`/:id/read`	Mark a notification as read

Dashboard — `/api/dashboard`

Method	Path	Description
GET	`/manager/:id`	Manager stats (projects, task counts, workers)
GET	`/worker/:id`	Worker stats (task counts, projects, completed today)

Invite — `/api/invite`

Method	Path	Description
POST	`/generate`	Manager generates a one-time invite link
POST	`/register`	Worker registers using an invite token (public)

4.6. Real-time Communication

The server exposes a Socket.IO endpoint authenticated via JWT. Upon connection each client joins a private room `user:{id}`. The REST `POST /api/messages` route saves the message to the database and then calls `emitNewMessage(receiverId, message)` to push the message instantly to the recipient's socket room without polling.

4.7. Security

- Passwords are stored as bcrypt hashes.
- All state-changing endpoints are protected by the JWT middleware (verified against `JWT_SECRET`).
- Google OAuth is handled server-side via Passport.js; no client secret is exposed to the frontend.
- File uploads are handled in-memory by Multer and stored as binary in the database (`bytea`), avoiding unauthenticated public file URLs.